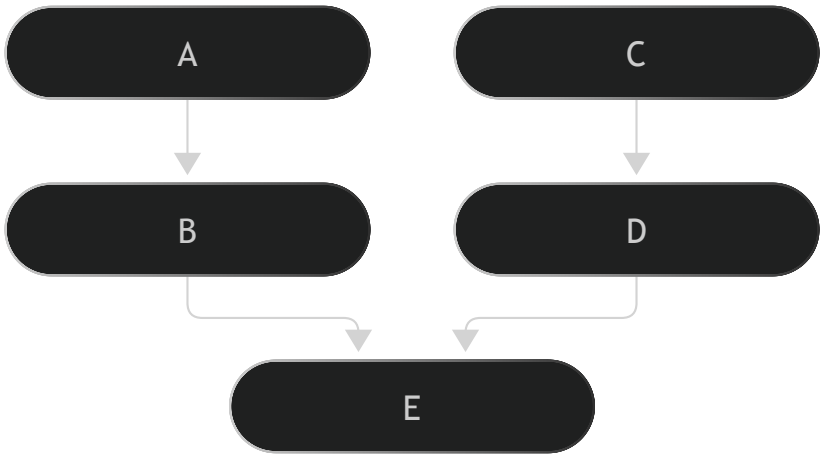
MRO

Порядок разрешения методов (Method Resolution Order) позволяет Python понять, из какого класса-предка нужно вызывать метод, если он не обнаружен непосредственно в классе-потомке. Если у каждого потомка есть только один предок, то задача тривиальна. Происходит восходящий поиск по всей иерархии. Если же используется множественное наследование, то можно столкнуться с определенными проблемами, которые будут рассмотрены позже.

В старых версиях Python порядок разрешения методов был достаточно примитивным: поиск вёлся во всех родительских классах слева направо на максимальную глубину. Т.е. если у родительского класса в свою очередь не было нужного метода, но были родители, то поиск производился в них по тем же правилам. Рассмотрим структуру:




```python
class A:
    def test(self):
        print("Вызван метод из класса A")

    def cooperative_show(self):
        print("-> A")
        super().cooperative_show()


class B(A):
    def test(self):
        print("Вызван метод из класса B")

    def cooperative_show(self):
        print("-> B")
        super().cooperative_show()

class C:
    def test(self):
        print("Вызван метод из класса C")

```

```
    def cooperative_show(self):
        print("-> C")

class D(C):
    def test(self):
        print("Вызван метод из класса D")

    def cooperative_show(self):
        print("-> D")
        super().cooperative_show()

class E(B, D):
    def cooperative_show(self):
        print("-> E")
        super().cooperative_show()
```

1. Просмотр порядка `MRO` для класса `E`

```python
for idx, cls in enumerate(E.mro(), start=1):
    print(f"{idx}. {cls.__name__}")
```

```
1. E
2. B
3. A
4. D
5. C
6. object
```

2. Поиск метода по цепочке `MRO`

```python
e = E()
e.test()
```

```
Вызван метод из класса B
```

3. Что будет, если удалить метод `test` из класса `B`

```python
del B.test
e.test()
```

```
Вызван метод из класса A
```

4. Кооперативный вызов через `super()`
   Цепочка вызовов проходит строго по MRO:

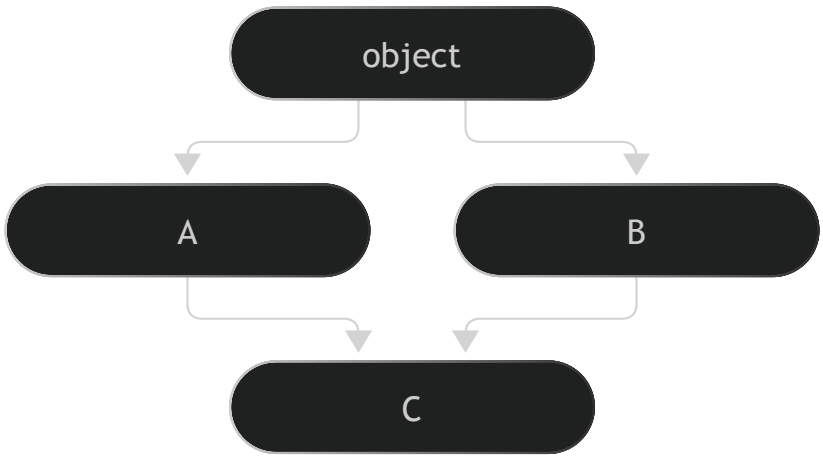
```python
e.cooperative_show()
```

```
-> E
-> B
-> A
-> D
-> C
```

При обращении к методу экземпляра класса E такой алгоритм произвёл бы поиск последовательно в классах E, B, A, D и C. Таким образом поиск вёлся сначала в первом классе-родителе и во всех его предках, затем во втором классе-родителе со всеми предками и т. д.

Однако, начиная с версии 2.2, появился базовый класс object, от которого рекомендовалось наследовать все пользовательские классы. Начиная с версии 3.0, старые классы больше не поддерживаются, а все пользовательские классы по умолчанию происходят от класса object.

Это породило проблему «ромбовидной структуры» («diamond diagram»).




```python
class Base:
    def save(self):
        return "Дефолтный метод из Base (или object)"

class A(Base):
    pass

class B(Base):
    def save(self):
        return "Важная логика из класса B!"

class C(A, B):
    pass
```

Эмулирует старый алгоритм поиска (до Python 2.2): Идет строго в глубину по первой (левой) ветке до самого верха (Base/object), и только потом переходит ко второй ветке.

```python
def old_dlr_search(cls, method_name):
    order = []

    def dfs(current_cls):
        order.append(current_cls)
        if method_name in current_cls.__dict__:
            return current_cls
        for base in current_cls.__bases__:
```

```
9               res = dfs(base)
10              if res:
11                  return res
12          return None
13
14      found_in = dfs(cls)
15      return found_in, order
```

1. Как работал СТАРЫЙ алгоритм (DLR)

```python
found_cls, search_path = old_dlr_search(C, "save")
path_str = " -> ".join([cls.__name__ for cls in search_path])
```

Цепочка поиска:

```python
path_str
```

```
'C -> A -> Base'
```

Результат: Найден метод из

```python
found_cls.__name__
```

```
'Base'
```

ОШИБКА: Класс B был полностью ПРОПУЩЕН, более специфичный метод не вызвался!

2. Как работает СОВРЕМЕННЫЙ Python 3 (Алгоритм C3)

```python
mro_str = " -> ".join([cls.__name__ for cls in C.mro()])
mro_str
```

```
'C -> A -> B -> Base -> object'
```

```python
c = C()
```

Результат вызова c.save():

```python
c.save()
```

```
'Важная логика из класса B!'
```

УСПЕХ: Метод из класса B найден до вызова базового Base/object!

Вообще вопрос чуть более сложен, чем проблема ромба. В этом примере в целом нормально провести новый класс `class C(A, B)` нельзя. Потому что не ясно как делать линеаризацию. При этом по старому алгоритму все довольно очевидно: `[C, A, B, X, Y, O]`

```
   -----------
  |           |
  |    O      |
  |  /   \    |
   - X    Y  /
     |  / | /
```

```
7      | /  |/
8      A    B
9      \   /
10       ?
```

```python
O = object
class X(O): ...
class Y(O): ...
class A(X,Y): ...
class B(Y,X): ...
```

Так что сейчас такой код в целом выдаст исключение:

```
TypeError: Cannot create a consistent method resolution order (MRO) for bases X, Y
```

Если у нас есть классы A и B, от которых наследуется класс C, то при поиске метода по старому алгоритму (C, A, object, B) получается, что если метод не определён в классах C и A он будет извлечён из object, даже если он определён в B. Это создаёт определённые неудобства, т.к. в object определены многие магические методы, типа `__init__`, `__str__` и т.п. Но даже если object заменить на некий пользовательский класс D, то проблема останется — менее специфичный метод класса-предка может отработать вместо более специфичного метода класса-потомка.

```python
class D:
    def process(self):
        return "Базовая (устаревшая) обработка из D"

class A(D):
    pass

class B(D):
    def process(self):
        return "Специфичная (актуальная) обработка из B"

class C(A, B):
    pass
```

Сравнение поведения

1. По СТАРОМУ алгоритму ($C \to A \to D \to B$):
   Поиск нашел бы process() в классе D и вернул:

   ```
   Базовая (устаревшая) обработка из D
   ```

   (Логика класса B проигнорирована!)
2. По НОВОМУ алгоритму C3 MRO в Python 3 ($C \to A \to B \to D$):

   ```python
   C().process()
   ```

   ```
   'Специфичная (актуальная) обработка из B'
   ```

Итак, у нас на руках есть сам класс, список всех его предков и связей между ними. Из этих данных нам нужно построить упорядоченный список классов, в которых будет производиться поиск метода слева направо. Такой список называется линеаризацией класса. Для простоты возьмем структуру:

```
object
   |
   A
   |
   B
```

Тогда линеаризацией для класса B будет список `[B, A, object]`. Т.е. при вызове B.something() метод сначала будет искаться в классе B. Если он там не найден, то поиск продолжится в классе A. Если его нет и там, то поиск завершится в классе object. Перебрав все классы из линеаризации и не обнаружив нужного метода, Питон выкинет ошибку AttributeError.

Для решения проблемы ромбовидной структуры линеаризация должна быть монотонной. Это значит, что если в линеаризации некого класса C класс A следует за классом B (она имеет вид `[C, …, B, …, A]`), то и для любого его потомка D класс B будет следовать за A в его линеаризации (она будет иметь вид `[D, …, C, …, B, …, A]`). Как можно заметить, старый порядок разрешения методов не монотонен, т.к. в случае ромбовидной структуры для класса A линеаризация есть `[A, object]`, для класса B — `[B, object]`, но для класса C — `[C, A, object, B]`, что нарушает свойство монотонности по отношению к классу B.

Для удовлетворения свойству монотонности в этом случае подойдут две линеаризации: `[C, A, B, object]` и `[C, B, A, object]`. Очевидно, что они обе не нарушают монотонности ни в отношении класса A (т.к. object следует за A в обоих случаях), ни в отношении класса B (т.к. object следует за B в обоих случаях). Так какую из них выбрать? В этом случае наиболее интуитивно-понятный способ — посмотреть на определение класса C. Если класс объявлен как C(A, B), то разумно будет взять первую линеаризацию, поскольку в ней B следует за A. Если класс объявлен как C(B, A), то будет лучше взять вторую линеаризацию, в которой A следует за B.

Такой выбор определяется порядком локального старшинства (local precedence ordering). Свойство порядка локального старшинства требует соблюдения для классов-родителей в линеаризации класса-потомка того же порядка, что и при его объявлении. Например, если класс объявлен как D(A, B, C), то в линеаризации D класс A должен стоять раньше B, а класс B — раньше C.

> Ну тут ты и так понимаешь, что нужно привести практический пример для двух вариантов, где выбираемые реализации метода будет отличаться в зависимости от порядка наследования ==ПРИВЕДЕН ТЕПЕРЬ==

Python

```python
class Base:
    def process(self) -> str:
        return "Базовая обработка (Base)"


class A(Base):
    def process(self) -> str:
```

```python
        return "Быстрая обработка (Класс A)"


class B(Base):
    def process(self) -> str:
        return "Надежная обработка (Класс B)"


# Вариант 1: Объявление C1(A, B) -> Класс A имеет приоритет над B
class C1(A, B):
    pass


# Вариант 2: Объявление C2(B, A) -> Класс B имеет приоритет над A
class C2(B, A):
    pass


if __name__ == "__main__":
    print("=== 1. Вариант C1(A, B) ===")
    print("MRO для C1:", [cls.__name__ for cls in C1.mro()])
    # В MRO класса C1 класс A идет раньше B: ['C1', 'A', 'B', 'Base', 'object']
    obj1 = C1()
    print("Результат вызова obj1.process():", obj1.process())
    # Вызовется A.process(), так как A первым встречается в MRO

    print("\n" + "=" * 40 + "\n")

    print("=== 2. Вариант C2(B, A) ===")
    print("MRO для C2:", [cls.__name__ for cls in C2.mro()])
    # В MRO класса C2 класс B идет раньше A: ['C2', 'B', 'A', 'Base', 'object']
    obj2 = C2()
    print("Результат вызова obj2.process():", obj2.process())
```

```
=== 1. Вариант C1(A, B) ===
MRO для C1: ['C1', 'A', 'B', 'Base', 'object']
Результат вызова obj1.process(): Быстрая обработка (Класс A)

========================================

=== 2. Вариант C2(B, A) ===
MRO для C2: ['C2', 'B', 'A', 'Base', 'object']
Результат вызова obj2.process(): Надежная обработка (Класс B)
```

```python
    # Вызовется B.process(), так как B первым встречается в MRO
```
Python

Алгоритм порядка разрешения методов C3

> Как тебе идея написать реализацию алгоритма C3 на python? И рассказывать алгоритм с практическим применением > > Либо же рассказывать его с исходниками на Си. Просто надо будет их подужать, оставив только самое нужно нам > > На python будет нагляднее с точки зрения понятности > На C будет логичнее т.к. мы говорим про реализацию алгоритма в python так-то > > Исходник алгоритма: > https://github.com/python/cpython/blob/main/Objects/typeobject.c

Для достижения указанных выше целей в Python используется алгоритм C3. Он достаточно прост, однако для лучшего понимания введём следующие условные обозначения: `[C1, C2, … CN]` – список из элементов `C1, C2, … CN`. Соответственно, `[C]` — список из одного элемента `C`.

`L[C]` – линеаризация класса C. Важно помнить, что любая линеаризация есть список по определению. `merge(L[C1], L[C2], …, L[CN])` – объединение элементов линеаризаций `L[C1], L[C2], …, L[CN]` в список с помощью некоторого алгоритма. По сути, этот алгоритм должен упорядочить все классы из `L[C1], L[C2], …, L[CN]` и исключить дублирование классов в итоговом списке.

Алгоритм C3 представляет из себя набор следующих правил:

- Линеаризация класса C есть одноэлементный список из самого класса `C` плюс объединение линеаризаций его родителей и списка всех его родителей. В условных обозначениях это можно записать как `L[C] = [C] + merge(L[C1], L[C2], …, L[CN], [C1, C2, …, CN])`, если класс C был объявлен как `class C(C1, C2, …, CN)`. Надо отметить, что каждая линеаризация `L[CX]` начинается с класса `CX`, который был дополнительно приписан в конец списка объединения как непосредственный родитель класса `C`. Зачем это сделано станет ясно далее.
- Объединение линеаризаций происходит следующим образом:
  1. Берётся нулевой элемент из первой линеаризации `(L[C1][0])`.
  2. Этот элемент ищется во всех других линеаризациях (от `L[C2]` до `L[CN]`).
  3. Если этот элемент найден где-то вне начала списка (`L[CK][X] == L[C1][0], X != 0;` по сути это значит, что `L[C1][0]` является чьим-то предком), то алгоритм переходит к первому шагу, беря в качестве нулевого элемент из следующей линеаризации (`L[C2][0]`).
  4. Если элемент нигде не найден в позиции отличной от нулевой, то он добавляется в конец итогового списка линеаризации и удаляется из всех рассматриваемых списков (от `L[C1]` до `L[CN]`; один и тот же класс не может встречаться в итоговом списке дважды). Если после удаления элемента остались пустые списки — они исключаются из объединения. После этого алгоритм повторяется с самого начала (от нового элемента `L[C1][0]`), если он есть. Если его нет — объединение закончено.
  5. Если алгоритм дошёл до последнего элемента `L[CN]`, но ни один из нулевых элементов не удовлетворяет правилам, то линеаризация не возможна.

> Возможно стоит описать алгоритм в псевдокоде Lovelace, вместо python

```
функция L(C):
    если C — базовый класс (object):
        вернуть [object]

    родители = [C1, C2, ..., Cn]
```

```
    списки_для_объединения = [L(C1), L(C2), ..., L(Cn), родители]

    вернуть [C] + merge(списки_для_объединения)


функция merge(списки):
    результат = []

    пока списки не пусты:
        кандидат_найден = Ложь

        для каждого списка из списки:
            голова = список[0]

            # Проверяем, не входит ли голова в хвост (элементы с индекса 1+) любого другого списка
            если голова НЕ содержится_в_хвостах(списки):
                результат.добавить(голова)
                удалить_из_всех_списков(списки, голова)
                удалить_пустые_списки(списки)
                кандидат_найден = Истина
                прервать_цикл_для  # Начинаем поиск с первого списка заново

        # Если прошли по всем головам и не нашли подходящего кандидата:
        если НЕ кандидат_найден:
            выбросить Ошибку("TypeError: Невозможно построить непротиворечивый MRO")

    вернуть результат
```

Чтобы алгоритм стал понятнее, разберём несколько примеров. Первым делом посмотрим, что же теперь происходит с нашей ромбовидной структурой. Для неё мы получаем: `L[C] = [C] + merge(L[A], L[B], [A, B])` `L[A] = [A] + merge(L[object], [object])` `L[B] = [B] + merge(L[object], [object])` `L[object] = [object]` (вырожденный случай) > Хороший пример для разбора. На презентации имеет смысл на одной части экрана показывать

Наибольший интерес представляет процесс объединения, так что разберём его более подробно. В случаях `L[A]` и `L[B]` выражение `merge(L[object], [object]) = merge([object], [object])` раскрывается тривиально. Поскольку и первый и второй списки состоят из одного элемента `object`, по пункту 4 правила объединения результатом будет `[object]`.

Теперь разберём `L[C] = [C] + merge(L[A], L[B], [A, B]) = [C] + merge([A, object], [B, object], [A, B])`. Распишем наше объединение по правилам C3:

- Возьмем нулевой элемент первого списка — `A`.
- Поищем его во всех прочих списках, т.е. в `[B, object]` и `[A, B]`.
- Поскольку класс `A` не обнаружен в списке `[B, object]`, а в списке `[A, B]` является нулевым элементом, то мы добавляем его в итоговый список линеаризации `L = [] + [A] = [A]`.

После этого класс `A` нужно удалить изо всех списков в объединении. Получим `L[C] = [C] + [A] + merge([object], [B, object], [B])`.
- Снова возьмём нулевой элемент первого списка — `object`.
- Поищем его во всех других списках и обнаружим, что object является первым (не нулевым) элементом списка `[B, object]`. Как уже было сказано выше, по сути, это означает, что класс object предок класса B. Стало быть, сначала имеет смысл поискать более специфичный метод в классе B. Поэтому алгоритм справедливо возвращается к пункту 1.
- Снова возьмём нулевой элемент, но на этот раз уже из второго списка, т.е. `B` из списка `[B, object]`.
- Поищем его в других списках и обнаружим его только в третьем списке `[B]` в нулевой позиции, что нас вполне устраивает. Следовательно, добавим его в итоговый список, а затем удалим изо всех списков в объединении. Получим `L = [A] + [B] = [A, B]` и соответственно `L[C] = [C] + [A, B] + merge([object], [object])`.
- Полученное объединение `merge([object], [object])` уже рассматривалось выше. В итоге получим `L[C] = [C] + [A, B] + [object] = [C] + [A, B, object] = [C, A, B, object]`. Список `[A, B, object]` — это и есть результат нашего объединения `merge(L[A], L[B], [A, B])`.

> Вообще супер! После этого примера как раз стоит привести пример, который я оставлял выше и почему этот алгоритм не сработает на таком примере

При создании класса (во время выполнения инструкции `class`) интерпретатор автоматически рассчитывает линеаризацию с помощью алгоритма C3 и сохраняет её в кортеж в атрибуте класса `__mro__` (или `Class.mro()`). > Атрибут `__mro__` вполне себе можно изменить и мы будем это делать в метаклассах

Когда происходит вызов `instance.method()`, Python не выполняет алгоритм C3 заново. Он просто последовательно проходит по готовому кортежу `type(instance).__mro__` и берёт первый найденный метод.

Если вы хотите получить линеаризацию для самопроверки или каких-то других целей — используйте функцию `mro()` (например, `C.__mro__`). > Лучше использовать функцию `mro()`, которая под капотом делает ровно это же (смотрит `__mro__`, но в) ==поправил==

Функция `super()` работает не с «родительским классом», а **строго по порядку в MRO текущего экземпляра**. Именно поэтому в множественном наследовании `super().method()` передаёт управление «соседу» справа по цепочке MRO (кооперативный вызов методов).

Виртуальное наследование (из C++) не может быть включено в **MRO** (Method Resolution Order), потому что MRO строит **линейную** последовательность классов, тогда как виртуальное наследование создает **нелинейную** структуру данных в памяти (граф с разделяемым базовым подобъектом).

> Надо будет подвизать с рассказом об `__slots__` в классах. Потому что это как раз метод оптимизации, который хранит доступные атрибуты и методы. Что в свою очередь просчитывается благодаря mro

> В дальнейшем в блоке про метаклассы будем ломать алгоритм поиска атрибутов через `__getattribute__`, переопределяя его по своему. Сможем имитировать циклическое наследование с помощью `__subclasscheck__` и `__instancecheck__`